

PYTHON NOTES

Python Character Set

- Letters - A to Z, a to z
- Digits - 0 to 9
- Special symbol +, -, *, / etc
- White space - blank space, tab, carriage return, newline, form feed
- Other characters - python can use process all ASCII and unicode characters as part of data or literals.

Variables

A variable is a name given to a memory location in a program.

name = "Siddharth"

age = 23

marks = 98.23

Rules for identifiers

① Identifiers can be combination of uppercase and lowercase letters, digits or an underscore (_).

So ~~some~~ myVariable, variable_1, variable_for_print all are valid in python.

② An Identifier cannot start with digit.

So variable_1 ✓ 1 variable X

③ We cannot use special symbols like !, #, @, %, & etc in our identifiers.

④ Identifier can be of any length.

Data types

print (type (age))	<class 'int'>	Integers
print (type (pi))	<class 'float'>	float
print (type (complex.num))	<class 'Complex'>	
print (type (A))	<class 'boolean'>	Boolean
print (type (name))	<class 'str'>	String

Key words.

Key words are reserved in python.

and	else	in	return
as	except	is	True
assert	finally	lambda	try
break	false	nonlocal	with
class	for	None	while
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

Comments in python

Single line Comment

"""

multi line

Comment

"""

Type of operators:

An operator is a symbol that performs a certain operation between operands.

- Arithmetic operators (+, -, *, /, %, **, //)
- Relational / comparison operators (==, !=, >, <, >=, <=)
- Assignment operators (=, +=, -=, *=, /=, %=, **=)
- Logical operators (not, and, or)

Type Casting

a, b = 1, "2"

c = int(b)

sum = a + c

Input in python.

input() statement is used to accept value from user.

int(input())

float(input())

#2 String

String is data type that store sequence of characters.

Basic operation.

"hello" + "world" → "helloworld" (concatenation)

len(str)

(length of string)

Indexing

A p na collage

0 1 2 3 4 5 6 7 8 9 10 11

str = "Apna collage"

str[0] is 'A' and str[1] is 'p'

str[0] = 'B' # not allowed

Slicing

Accessing part of string.

str[starting_idx: ending_idx] # ending index is not included

str = "Apna collage"

str[1:4] is 'pna'

str[:4] is same as str[0:4] - 'apna'

str[1:] is same as str[1:len(str)]

Apple
-5-4-3-2-1

Negative indexing is possible

str = "Apple"

str[-5:-1] is Appl

String functions

str = "I am a Coder."

str.endswith("er") # return true if string ends with substr.

str.capitalize() # Capitalized 1st char

str.replace(old, new) # replace all occurrences of old with new

str.find("word") # return 1st index of 1st occurrence

str.count("am") # count the occurrence of substr in string

Conditional statements

if-elif-else (syntax)

if (condition):

statement 1

elif (condition):

statement 2

else:

statement N

#3

List in python

A built in data type that store ~~set~~ set of value.
It can store elements of different types (int, float, str etc).

```
marks = [87, 86, 88, 92, 95] # mark[0], mark[1]...
```

```
student = ["karan", 92, "Delhi"] # student[0], student[1]...
```

```
student[0] = "Arjun" # allowed in python
```

List slicing

Similar to string slicing

list.name [starting-idx: ending-idx] # ending idx is not included

```
marks = [87, 86, 85, 92, 95]
```

```
marks[1:4] is [86, 85, 92]
```

```
marks[:4] is as same mark[0:4]
```

```
marks[1:] is as same mark[1:len(marks)]
```

List Methods

```
list = [2, 1, 3]
```

```
list.append(4) # adds one element at end [2, 1, 3, 4]
```

```
list.sort() # sort in ascending order [1, 2, 3]
```

```
list.sort(reverse = true) # sort in descending order [3, 2, 1]
```

```
list.reverse() # reverse list [3, 1, 2]
```

```
list.insert(idx, el) # insert element at index.
```

```
list = [1, 2, 3, 1]
```

```
list.remove(1) # remove first occurrence of element [2, 3, 1]
```

```
list.pop() # remove element at idx
```


Tuple in Python.

A built in datatype that let us create immutable sequence of us

```

    tup = (87, 84, 83, 92, 53) # tup[0], tup[1]

```

```

    tup[0] = 43 # NOT allowed in python

```

Tuple methods

```

    tup = (2, 1, 3, 1)

```

```

    tup.index(1) # returns index of first occurrence tup.index(1) is 1

```

```

    tup.count(1) # count total occurrence tup.count(1) is 2

```

4 Dictionary in python

Dictionary are used to store data value in key:value pairs

They are unordered, mutable (changeable) & don't allow duplicate key

```

    dict = {

```

```

        "name": "Shradha",

```

```

        "Cpga": 9.6,

```

```

        "marks": [90, 97, 95],

```

```

    }

```

```

    dict["name"], dict["Cpga"], dict["marks"]

```

```

    dict["key"] = "value" # to assign or add new

```

Nested dictionaries

```

    studen = {

```

```

        "name": "sid",

```

```

        "score": {

```

```

            "chem": 90,

```

```

            "phy": 97,

```

```

        }

```

```

    }

```

```

    Student["score"]["chem"]

```


Dictionary methods

my Dict. keys() # return all keys
my Dict. values() # return all values
my Dict. items() # return all (key, val) pair in tuples
~~my Dict. get()~~
my Dict. get("key") # return the key according to value
my Dict. update(new Dict) # insert the specified items to the dictionary

Set in python

Set is the collection of the unordered items

Each element in set must be unique & immutable

nums = {1, 2, 3, 4}

set 2 = {1, 2, 2, 2} # all repeated element store only once

So set 2 = {1, 2}

null set = set() # empty set syntax

set methods

set.add(e1) # add an element

set.remove(e1) # remove the element

set.clear() # empties the set

set.pop() # remove a random value

set.union(set2) # combine both set value & return new set

set.intersection(set2) # combine common set value & return new set

(set2, set & both) appear

#5

Loops in python

Loops are used to repeat instruction.

- While loops.

while

while condition:

some work.

Break: used to terminate the loop when encountered.

Continue: terminates execution in the current iteration &

continues execution of the loop with the next iteration

Loops are used for sequential traversal: list, string tuples etc.

- for loop

for el in list:

some work

for loop with else statement

for el in list:

some work

else:

work when loop end

Range ()

range function returns a sequence of numbers starting from 0 by default) and increment by 1 (by default) stop before the specified number.

range (start? stop, step?)


```
for el in range(5):
    print(el)
```

output (0, 1, 2, 3, 4, 5)

```
for el in range(1, 5):
    print(el)
```

output (1, 2, 3, 4, 5)

```
for (el in range(1, 5, 2)):
    print(el)
```

output (1, 3, 5)

Pass statement

pass is a null statement that does nothing. It is used as a placeholder for future code.

```
for el in range(10):
    pass
```

#6

Function in python

Block statement that perform a specific task!

```
def func_name(param1, param2): - # function definition
    # some work
```

```
    return val
```

```
func_name(arg1, arg2) - # function call
```

```
def sum(a, b):
```

```
    s = a + b
```

```
    return s
```

```
print(sum(2, 3))
```


Default Parameters.

Assigning a default value to parameter, which is used when no argument is passed.

Recursion.

When a function calls itself repeatedly.

① # prints n to 1 backward. ② # return n!

```
def show(n):
```

```
    if (n==0):
```

```
        return
```

```
    print(n)
```

```
    show(n-1)
```

```
def fact(n):
```

```
    if (n==0 or n==1):
```

```
        return 1
```

```
    else:
```

```
        return n * fact(n-1)
```

7 File Input/output in python

Python can be used to perform operations on a file. (read & write data)

• Type of file

① Text file - .txt, .docx, .log etc

② Binary file - .mp4, .mov, .png, .jpeg etc

open read and close file

We have to open a file before reading or writing.

```
f = open("file name", "mode")
```

sample.txt

demo.docx

r: read mode

w: write mode

```
data = f.read()
```

```
f.close()
```


Character

Meaning

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, truncating the file first
'x'	create a new file and open it for writing
'a'	open for writing, appending to the end of the file if it exists.
'b'	binary mode
't'	text mode (default)
'+'	open a disk file for updating (reading or writing)

Reading a file

```
data = f.read()
```

reads entire file

```
data = f.readline()
```

reads one line at a time

Writing to a file

```
f = open("demo.txt", "w")  
f.write("this is a line")
```

overwrites the entire file

```
f = open("demo.txt", "a")  
f.write("this is a new line")
```

adds to the file

With syntax

```
with open("demo.txt", "a") as f:  
    data = f.read()
```

Deleting a file

Using the os module

Module (like a ~~code~~ library) is a file written by another programmer
the generally has a function we can use

```
import os
```

```
os.remove(file name)
```


Package

Module in Python.

①

```
import pyjokes
joke = pyjokes.get_joke()
print(joke)
```

appstring

②

```
name = input("Enter your name")
print(f"Good morning {name}")
```

③

```
letter = ""
Dear <|name|>,
You are Selected!
<|Date|>
print(letter.replace("<|name|>", "Siddhu").replace("<|Date|>", "29 Jan 2006"))
```

④ Temp change

```
def f_to_c(f):
    return 5*(f-32)/9
f = int(input("Enter Temp in F: "))
c = f_to_c(f)
print(f" {round(C,2)} °C")
```

⑤ Sum by recursion

```
def sum(n):
    if (n==1):
        return 1
    return sum(n-1)+n
print(sum(4))
```

(sum of 1 to 4) = 10

OOPs with Python.

1. Class Employee:

language = "Py"

salary = 1200000

Sid = Employee()

// language and salary are

print(~~han~~ Sid.language, ~~han~~ Sid.salary) // class attribute

Rohan = Employee()

print(Rohan.salary, Rohan.language)

a = Employee()

// name is the object attribute

a.name = "King"

print(a.name)

Object (instance) > class attribute

self

• Class AB:

name = "Sid"

age = 28

def show(self):

print(f"The name is {self.name} and age {self.age}")

a = AB()

a.show()

⑥ staticmethod: ~~Don't use~~ for if you want to not use self.
__init__: Dunder method which is called automatically.

★ Class AB:

age = 12

@staticmethod

def great():

print("Good morning");

a = AB()

a.great()

Constructor.

Class Sid:

name = "Siddu"

age = 19

def __init__(self, name, age):

self.name = name

self.age = age

print("I am a king")

s = Sid("Siddu", 20)

S = Sid("Siddharth", 20)

print(s.name, s.age)

Note: When we use instance (object) attribute but class attribute not change.